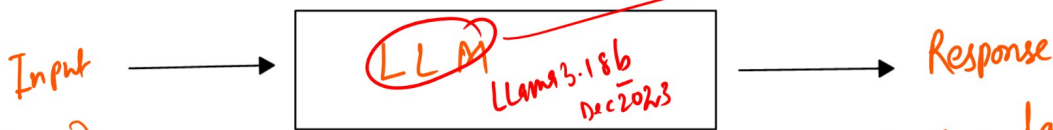


- Agenda:-
- ① Langchain Tools
 - ② AI Agents vs Agentic AI
 - ③ LangGraph & key components
 - ④ Chatbot with LangGraph

Workflow $\Rightarrow$ Step by step. Instruction

Tools $\Rightarrow$ Already capable but doesn't have real time info

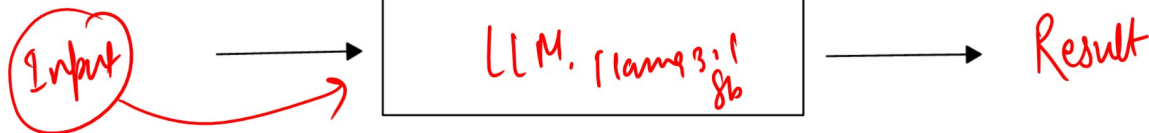
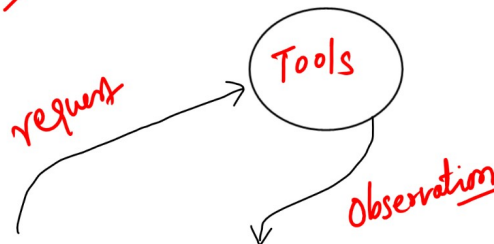


Any GenAI Application chatGPT

Who won the loss in IPL 2026 final?

Langchain $\Rightarrow$ DAG $\Rightarrow$ Directed Acyclic Graph $\Rightarrow$ Langchain $\Rightarrow$ Simple workflow $\Rightarrow$ Linear execution

Real world scenarios: Loop, Condition, branching $\Downarrow$ LangGraph



Langchain

formulate final response

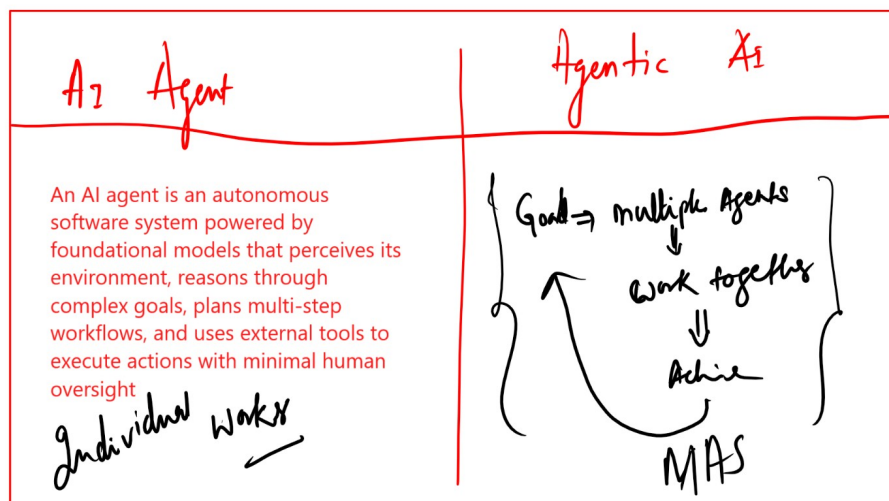
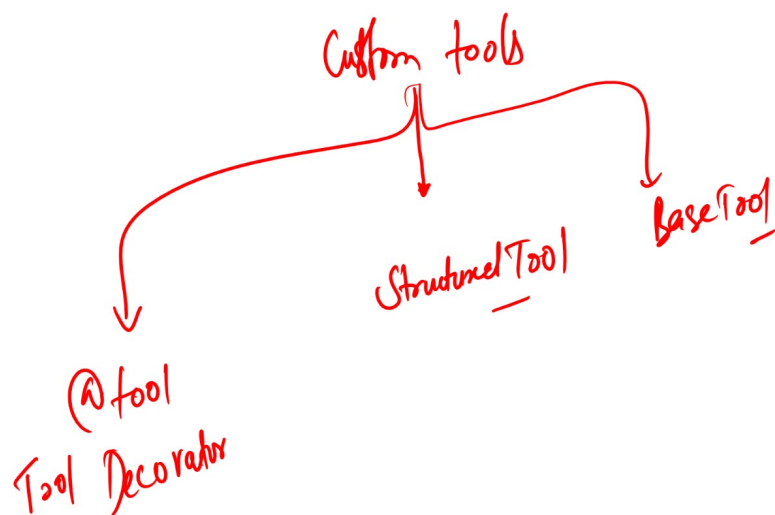
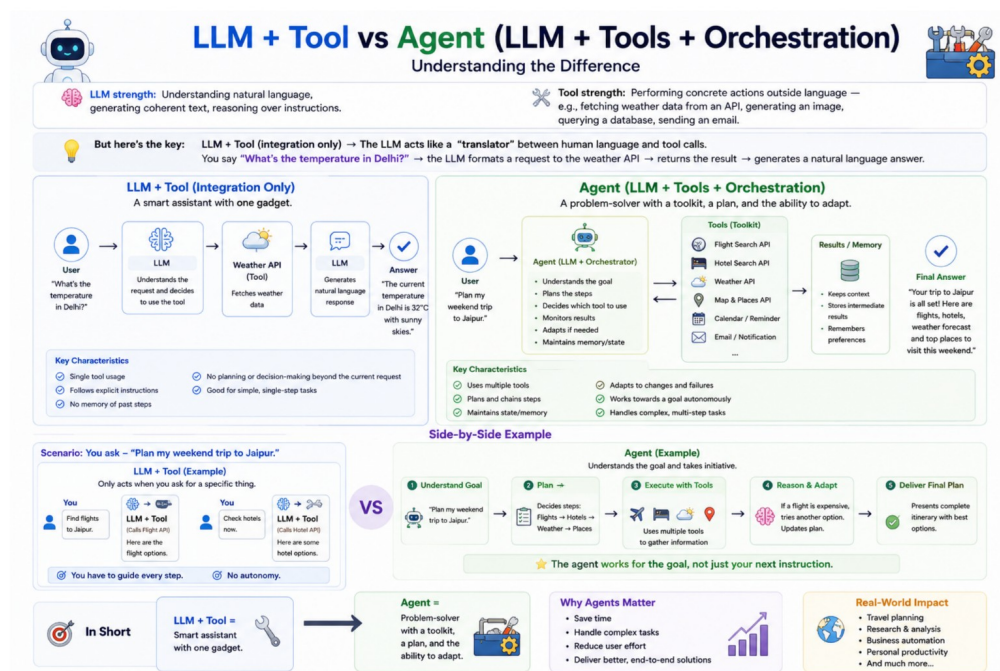
$\Rightarrow$ Internet Search $\Rightarrow$ DuckDuckGo Search, Tavily, Brave, Google... | Shell, Arxiv | Wikipedia, Github

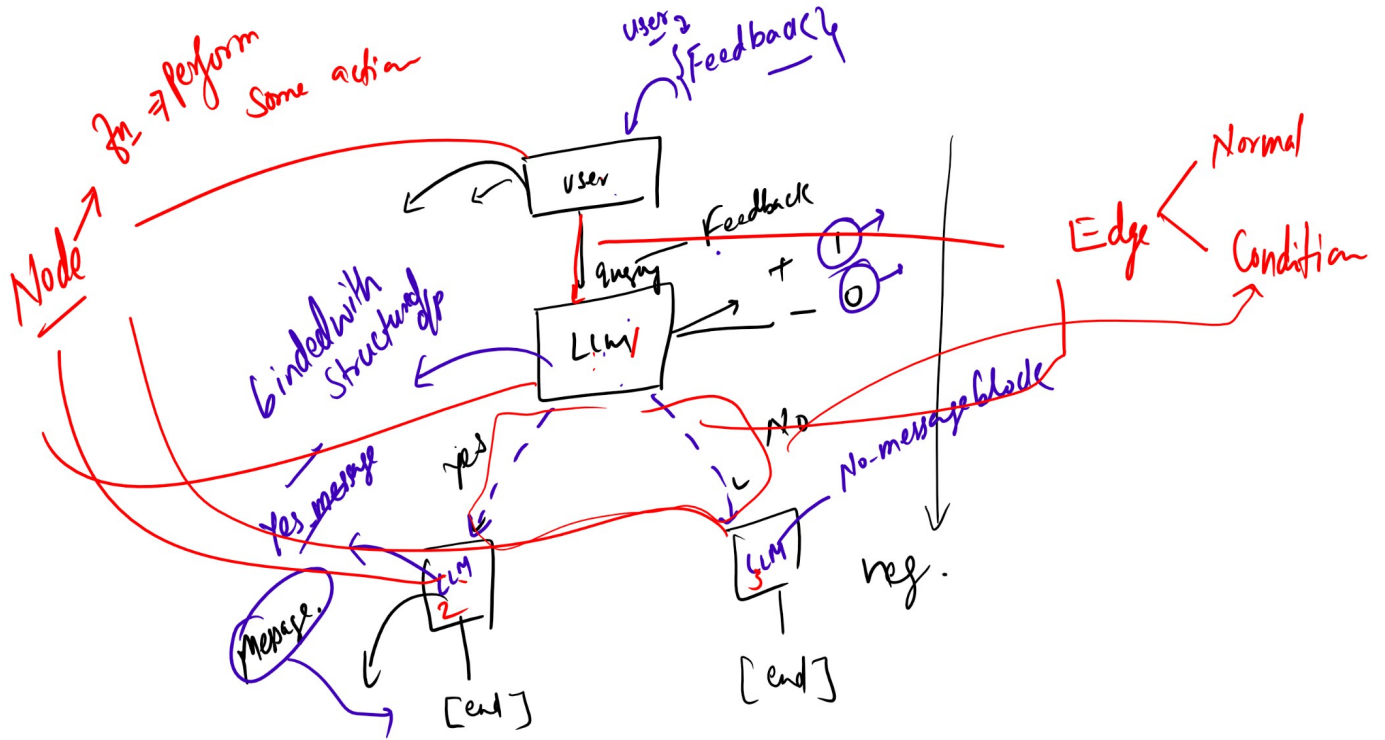
LLM + Tool (single connection):

The LLM can call the tool when prompted, but it's essentially just an interface — it doesn't decide when or why to use the tool beyond the immediate instruction. Think of it like a calculator attached to a chatbot: it can compute when asked, but it won't autonomously plan or reason about when to use it.

Tool Calling LLM.

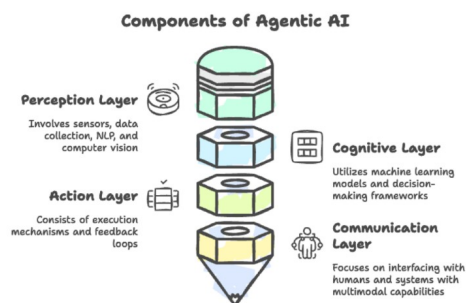
Most people think of AI as just a chatbot that answers questions. But the real future is Agentic AI – systems that can reason, plan, use tools, and act autonomously. Among these, tool-use is one of the most critical capabilities. Without tools, even the best LLM is limited to what it has seen in training data. With tools, an AI system can fetch live data, make API calls, query databases, or even control real applications. This makes the difference between a chatbot and a true AI assistant.





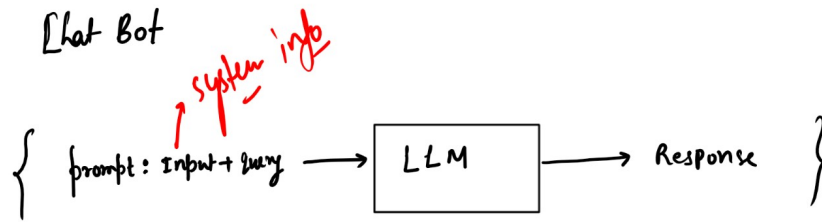
Key Concept :- Terminology that will be used in langgraph workflows.

1. Node ✓
2. Edge ✓
 - 2.1 Normal / Data flow edge ✓
 - 2.2 Conditional edges ✓
3. state ⇒ Hold the information ⇒ mutable object ⇒ data schema ✓
4. State Graph



Let's understand a simple workflow :-

eg:- Chat Bot

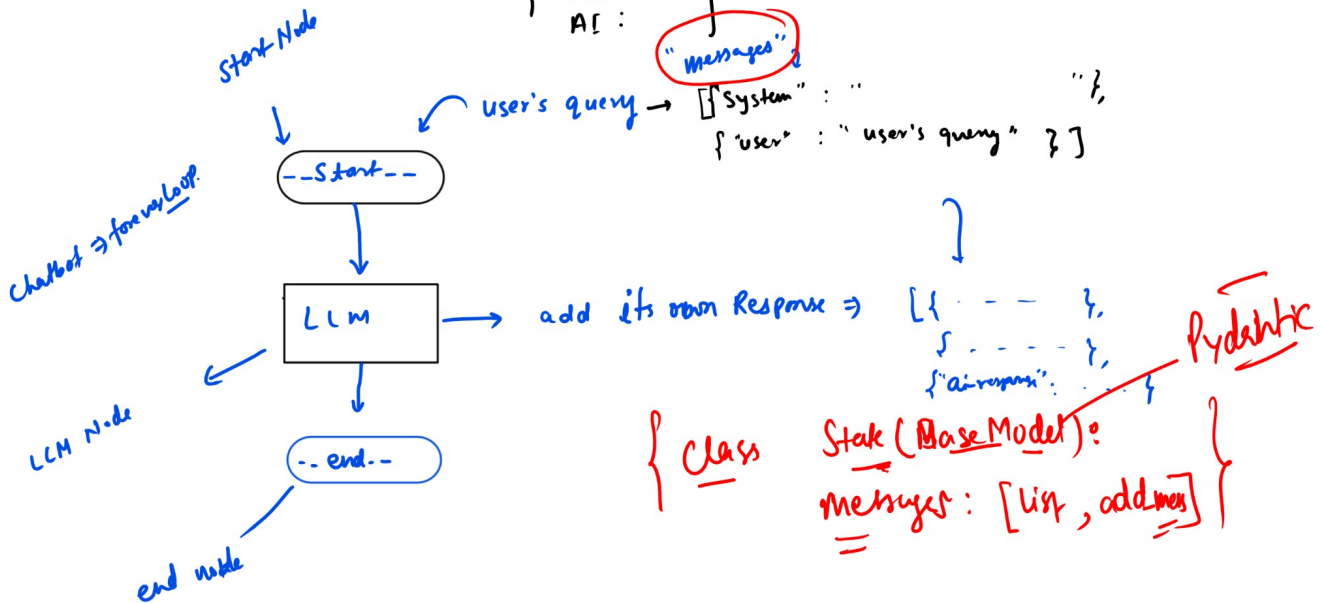


so, the workflow:- Input → LLM → Response

As it is chatbot ⇒ There will be multi-turn conversation

System
User:-
AI:
User:
AI:

⇒ list of dictionary ⇒ information



At each Node ⇒ fn ⇒ performs some task

Takeaway:- ① each node is a fn

① every node ⇒ needs a info

↓
we need to store this
somewhere

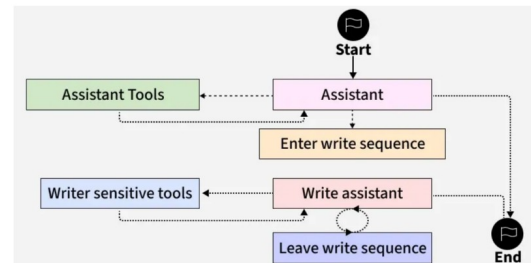
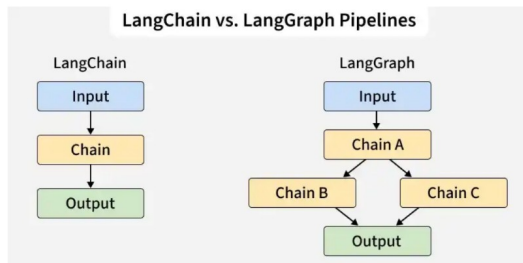
Pydantic/TypedDict

State ⇒ carry the
info. as like
Convey.
↓
Some node adds
some info some
removed.

It is required
to make a state schema
↓
that has its info

⇒ To follow info
we need data
flow pipe ⇒ edge.

Langgraph :- Product from Langchain
 ↓
 Graph Based Agentic framework }



<https://docs.langchain.com/oss/python/langgraph/overview>

Installation ⇒ `uv add langgraph`

or write langgraph in requirements.txt
 ↳ execute command in terminal ⇒ `uv add -r requirements.txt`

Whenever ⇒ we build any workflow
 ↓

we import ⇒ StateGraph, START, END from langgraph-graph

StateGraph →

This is the core class that lets you define a directed graph of nodes (steps) and edges (transitions). Each node represents a unit of work (like calling a tool, reasoning with the LLM, or transforming data). The graph manages how state (messages, variables, tool outputs) flows between nodes.

Start →

A special marker node that represents the entry point of your workflow. It's where execution begins when you run the graph. You connect Start to the first node in your workflow.

End →

A special marker node that represents the exit point of your workflow. Once execution reaches End, the workflow is complete. You connect the final node(s) to End.

① First we will understand "How to create simple workflows in LangGraph"

